

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
про виконання лабораторної роботи № 3
« Ітератори, генератори та потокова обробка даних »

Виконала:
студентка 3 курсу
групи ФЕП-31
Антошкова Аліна Олександрівна
Перевірів:
Доц. Куньо Іван Михайлович

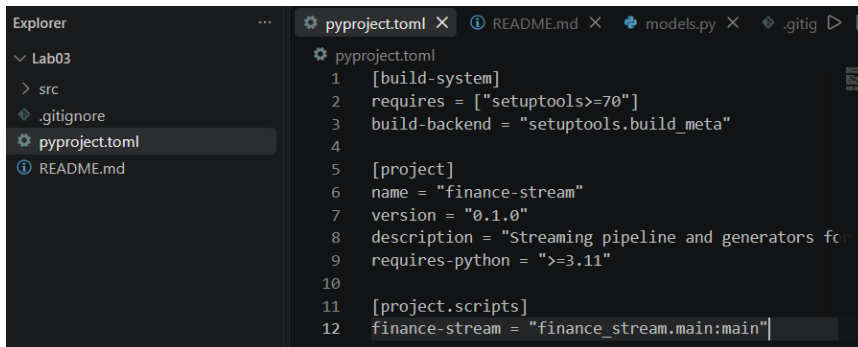
Мета: Сформуванати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних.

Варіант 12

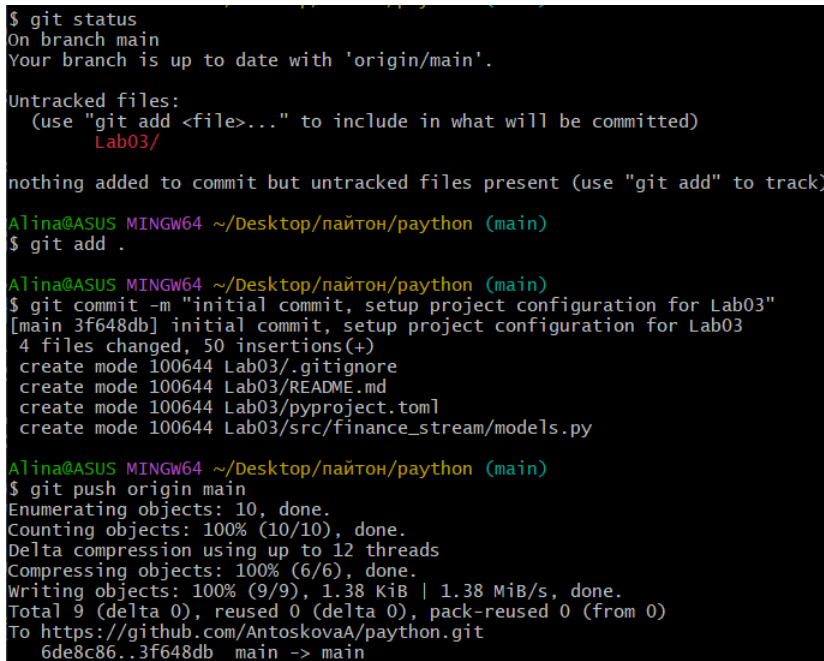
Постановка завдання:

Створити потоковий конвеєр (streaming pipeline) для обробки журналу фінансових операцій. Реалізувати потокове валідування записів, фільтрацію income/expense, потокове обчислення загальних доходів і витрат, кумулятивний баланс через itertools.accumulate, фільтрацію витрат за категорією та порогом суми, підрахунок витрат за категоріями, batch-обробку, вибірку перших N операцій через itertools.islice, порівняння eager та lazy підходів на 10 000, 100 000 та 500 000 записах із замірами часу та пікової пам'яті (tracemalloc).

1. Додавання файлів базової конфігурації (.gitignore, pyproject.toml, README.md) та виконання початкового коміту для Lab03



```
pyproject.toml
1 [build-system]
2   requires = ["setuptools>=70"]
3   build-backend = "setuptools.build_meta"
4
5 [project]
6   name = "finance-stream"
7   version = "0.1.0"
8   description = "Streaming pipeline and generators for financial data"
9   requires-python = ">=3.11"
10
11 [project.scripts]
12   finance-stream = "finance_stream.main:main"
```



```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  Lab03/

nothing added to commit but untracked files present (use "git add" to track)
Alina@ASUS MINGW64 ~/Desktop/найтон/python (main)
$ git add .
Alina@ASUS MINGW64 ~/Desktop/найтон/python (main)
$ git commit -m "initial commit, setup project configuration for Lab03"
[main 3f648db] initial commit, setup project configuration for Lab03
4 files changed, 50 insertions(+)
create mode 100644 Lab03/.gitignore
create mode 100644 Lab03/README.md
create mode 100644 Lab03/pyproject.toml
create mode 100644 Lab03/src/finance_stream/models.py
Alina@ASUS MINGW64 ~/Desktop/найтон/python (main)
$ git push origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 1.38 KiB | 1.38 MiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AntoskovaA/python.git
6de8c86..3f648db main -> main
```

2. Налаштування віртуального оточення та створення структури моделей даних і власного ітератора

```

src > finance_stream > models.py > TransactionLimitIterator > __next__
1  from typing import NamedTuple
2
3  class TransactionRecord(NamedTuple):
4      """Незмінна модель операції для передачі через потоковий конвеєр"""
5      date: str
6      category: str
7      amount: float
8      op_type: str # "income" або "expense"
9      description: str
10
11 class TransactionLimitIterator:
12     """Власний клас-ітератор відповідно до протоколу __iter__ та __next__"""
13
14     def __init__(self, transactions: list[TransactionRecord], max_count: int):
15         self.data = transactions
16         self.limit = min(max_count, len(transactions))
17         self.index = 0
18
19     def __iter__(self) -> "TransactionLimitIterator":
20         return self
21
22     def __next__(self) -> TransactionRecord:
23         if self.index >= self.limit:
24             raise StopIteration
25         record = self.data[self.index]
26         self.index += 1
27         return record

```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab03 (main)
$ git add src/finance_stream/__init__.py src/finance_stream/models.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab03 (main)
$ git commit -m "created virtual environment and models with custom iterator"
[main 7d49f2c] created virtual environment and models with custom iterator
 2 files changed, 2 insertions(+), 1 deletion(-)
 create mode 100644 Lab03/src/finance_stream/__init__.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab03 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 651 bytes | 651.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/AntoskovaA/python.git
 3f648db..7d49f2c  main -> main
```

```
models.py X readers.py X
src > finance_stream > readers.py > read_lines
1 from collections.abc import Iterator
2 from pathlib import Path
3
4
5 def read_lines(file_path: Path) -> Iterator[str]:
6     """Потокове читання файлу рядок за рядком через yield from."""
7     with file_path.open("r", encoding="utf-8") as file:
8         yield from file
```

```
pipeline.py X parsers.py X
> finance_stream > parsers.py > parse_csv_rows
1 import csv
2 from collections.abc import Iterable, Iterator
3
4
5 def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
6     """Потоковий розбір рядків за допомогою DictReader без завантаження в пам'ять."""
7     reader = csv.DictReader(lines)
8     yield from reader
```

```
pipeline.py X parsers.py X
src > finance_stream > pipeline.py > build_finance_pipeline
1 from collections.abc import Iterator
2 from pathlib import Path
3 from finance_stream.filters import normalize_transactions, validate_transactions
4 from finance_stream.models import TransactionRecord
5 from finance_stream.parsers import parse_csv_rows
6 from finance_stream.readers import read_lines
7
8
9 def build_finance_pipeline(file_path: Path) -> Iterator[TransactionRecord]:
10     """Збирання повного лінивого конвеєра обробки."""
11     lines = read_lines(file_path)
12     rows = parse_csv_rows(lines)
13     valid_records = validate_transactions(rows)
14     normalized = normalize_transactions(valid_records)
15     return normalized
```

```
pipeline.py X parsers.py X filters.py X
> finance_stream > filters.py > filter_expenses_by_threshold
1 from collections.abc import Iterable, Iterator
2 from finance_stream.models import TransactionRecord
3
4
5 def validate_transactions(rows: Iterable[dict[str, str]]) -> Iterator[TransactionRecord]:
6     """Валідація значень та створення типізованих записів."""
7     for row in rows:
8         try:
9             date_val = row["date"].strip()
10            category = row["category"].strip()
11            amount = float(row["amount"])
12            op_type = row["type"].strip().lower()
13            description = row.get("description", "").strip()
14        except (ValueError, KeyError, TypeError):
15            continue
16
17        if not date_val or not category or amount <= 0:
18            continue
19        if op_type not in ("income", "expense"):
20            continue
21
22        yield TransactionRecord(
23            date=date_val,
24            category=category,
25            amount=amount,
26            op_type=op_type,
27            description=description,
28        )
```

```

def filter_by_type(records: Iterable[TransactionRecord], op_type: str) -> Iterator[TransactionRecord]:
    """Фільтрація за типом операції."""
    for record in records:
        if record.op_type == op_type:
            yield record

def filter_expenses_by_threshold(
    records: Iterable[TransactionRecord], category: str, threshold: float
) -> Iterator[TransactionRecord]:
    """Фільтрація витрат за категорією та мінімальною сумою."""
    for record in records:
        if record.op_type == "expense" and record.category == category and record.amount >= threshold:
            yield record

```

```

pipeline.py X parsers.py X batches.py X
> finance_stream > batches.py > batched_transactions
1 from collections.abc import Iterable, Iterator
2 from itertools import islice
3 from finance_stream.models import TransactionRecord
4
5
6 def batched_transactions(
7     records: Iterable[TransactionRecord], batch_size: int
8 ) -> Iterator[list[TransactionRecord]]:
9     """Розбиття потоку на порції (батчі) за допомогою itertools.islice."""
10    iterator = iter(records)
11    while True:
12        batch = list(islice(iterator, batch_size))
13        if not batch:
14            return
15        yield batch

```

```

pipeline.py X parsers.py X analytics.py X
> finance_stream > analytics.py > stream_cumulative_balance
1 from collections import Counter
2 from collections.abc import Iterable, Iterator
3 from itertools import accumulate
4 from finance_stream.models import TransactionRecord
5
6
7 def stream_financial_summary(records: Iterable[TransactionRecord]) -> dict:
8     """Агрегація балансу та витрат за один прохід по потоку."""
9     total_income = 0.0
10    total_expense = 0.0
11    category_expenses: Counter = Counter()
12
13    for r in records:
14        if r.op_type == "income":
15            total_income += r.amount
16        elif r.op_type == "expense":
17            total_expense += r.amount
18            category_expenses[r.category] += r.amount
19
20    return {
21        "total_income": total_income,
22        "total_expense": total_expense,
23        "balance": total_income - total_expense,
24        "category_expenses": dict(category_expenses),
25    }
26
27 def stream_cumulative_balance(records: Iterable[TransactionRecord]) -> Iterator[float]:
28     """Обчислення кумулятивного балансу за допомогою itertools.accumulate."""
29     signed_amounts = (
30         r.amount if r.op_type == "income" else -r.amount for r in records
31     )
32     yield from accumulate(signed_amounts)

```

```

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git add src/finance_stream/readers.py src/finance_stream/parsers.py src/finance_stream/filters.py src/finance_stream/transformations.py src/finance_stream/batches.py src/finance_stream/analytics.py src/finance_stream/pipeline.py

```

```

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git commit -m "implement stream processing generators, pipeline and itertools analytics"
[main 4ffe38c] implement stream processing generators, pipeline and itertools analytics
9 files changed, 160 insertions(+), 2 deletions(-)
create mode 100644 Lab03/src/finance_stream/analytics.py
create mode 100644 Lab03/src/finance_stream/batches.py
create mode 100644 Lab03/src/finance_stream/filters.py
create mode 100644 Lab03/src/finance_stream/parsers.py
create mode 100644 Lab03/src/finance_stream/pipeline.py
create mode 100644 Lab03/src/finance_stream/readers.py
create mode 100644 Lab03/src/finance_stream/transformations.py

```

```

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git push origin main
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 12 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (14/14), 3.56 KiB | 1.78 MiB/s, done.
Total 14 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/AntoskovaA/paython.git
7d49f2c..4ffe38c main -> main

```

4. Створення точки входу програми, генератора даних та модуля експериментальних замірів пам'яті

```
analytics.py X generate_data.py X
c > finance_stream > generate_data.py > ...
1 import csv
2 import random
3 from pathlib import Path
4
5 INCOME_CATS = ("Зарплата", "Фриланс", "Інвестиції", "Кешбек")
6 EXPENSE_CATS = ("Продукти", "Транспорт", "Комунальні", "Розваги", "Одяг", "Здоров'я")
7
8
9 def generate_dataset(file_path: Path, count: int) -> None:
10     file_path.parent.mkdir(parents=True, exist_ok=True)
11     with file_path.open("w", encoding="utf-8", newline="") as file:
12         writer = csv.writer(file)
13         writer.writerow(["date", "category", "amount", "type", "description"])
14         for i in range(1, count + 1):
15             if random.random() < 0.3:
16                 op_type = "income"
17                 cat = random.choice(INCOME_CATS)
18                 amt = round(random.uniform(500, 30000), 2)
19             else:
20                 op_type = "expense"
21                 cat = random.choice(EXPENSE_CATS)
22                 amt = round(random.uniform(50, 4000), 2)
23
24             writer.writerow([
25                 f"2026-09-{(i % 28) + 1:02d}",
26                 cat,
27                 amt,
28                 op_type,
29                 f"Операція №{i}",
30             ])
31
32
33 if __name__ == "__main__":
34     generate_dataset(Path("data/transactions_sample.csv"), 1000)
35     print("Згенеровано базовий датасет data/transactions_sample.csv")
```

```
analytics.py X generate_data.py X experiment.py X
> finance_stream > experiment.py > ...
1 import csv
2 from pathlib import Path
3 from time import perf_counter
4 import tracemalloc
5 from finance_stream.generate_data import generate_dataset
6 from finance_stream.pipeline import build_finance_pipeline
7
8
9 def run_eager(file_path: Path) -> int:
10     with file_path.open("r", encoding="utf-8") as f:
11         reader = csv.DictReader(f)
12         all_records = list(reader) # Повне завантаження всіх записів у пам'ять
13         return len([r for r in all_records if r["type"] == "expense"])
14
15
16 def run_lazy(file_path: Path) -> int:
17     pipeline = build_finance_pipeline(file_path)
18     count = 0
19     for r in pipeline:
20         if r.op_type == "expense":
21             count += 1
22     return count
23
24
25 def measure(func, *args) -> tuple[float, float]:
26     tracemalloc.start()
27     t0 = perf_counter()
28     func(*args)
29     elapsed = perf_counter() - t0
30     _, peak = tracemalloc.get_traced_memory()
31     tracemalloc.stop()
32     return elapsed, peak / (1024 * 1024)
```

```
def main() -> None:
    counts = [10_000, 100_000, 500_000]
    print(f"{'Records':<10} | {'Eager time':<12} | {'Lazy time':<12} | {'Eager Peak MB':<14} | {'Lazy Peak MB':<14}")
    print("-" * 72)
    for c in counts:
        p = Path(f"data/test_{c}.csv")
        generate_dataset(p, c)

        t_eag, mem_eag = measure(run_eager, p)
        t_laz, mem_laz = measure(run_lazy, p)

        print(f"{c:<10} | {t_eag:<10.4f} s | {t_laz:<10.4f} s | {mem_eag:<12.2f} MB | {mem_laz:<12.4f} MB")
        p.unlink()

if __name__ == "__main__":
    main()
```

```
ytics.py X experiment.py X main.py X
finance_stream > main.py > ...
from itertools import islice
from pathlib import Path
from finance_stream.analytics import stream_cumulative_balance, stream_financial_summary
from finance_stream.batches import batched_transactions
from finance_stream.filters import filter_expenses_by_threshold
from finance_stream.generate_data import generate_dataset
from finance_stream.models import TransactionLimitIterator
from finance_stream.pipeline import build_finance_pipeline

def main() -> None:
    csv_file = Path("data/transactions_sample.csv")
    if not csv_file.exists():
        generate_dataset(csv_file, 1000)

    print("=== Лабораторна робота № 3: Поточковий конвеєр (Варіант 12) ===")

    # 1. Перші 5 елементів через itertools.islice
    print("\n--- Перші 5 операцій (itertools.islice) ---")
    p1 = build_finance_pipeline(csv_file)
    for tx in islice(p1, 5):
        print(f"{tx.date} | {tx.op_type.upper():7} | {tx.category:12} | {tx.amount:8.2f} грн | {tx.description}")

    # 2. Generator expression
    p2 = build_finance_pipeline(csv_file)
    total_exp_gen = sum(t.amount for t in p2 if t.op_type == "expense")
    print(f"\n[Generator expression] Загальні витрати: {total_exp_gen:.2f} грн")

    # 3. Фільтрація витрат за порогом
    p3 = build_finance_pipeline(csv_file)
    large_food = list(islice(filter_expenses_by_threshold(p3, "Продукти", 1500.0), 3))
    print(f"\n[Filter & islice] Витрати на 'Продукти' >= 1500 грн (знайдено {len(large_food)})")
```

```
def main() -> None:

    # 4. Батчинг
    p4 = build_finance_pipeline(csv_file)
    batch_gen = batched_transactions(p4, batch_size=250)
    first_batch = next(batch_gen)
    print(f"\n[Batching] Розмір першого батча: {len(first_batch)} записів")

    # 5. Кумулятивний баланс (itertools.accumulate)
    p5 = build_finance_pipeline(csv_file)
    first_cum = list(islice(stream_cumulative_balance(p5), 5))
    print(f"\n[itertools.accumulate] Кумулятивний баланс перших 5 операцій:", [round(x, 2) f

    # 6. Власний ітератор
    p6 = build_finance_pipeline(csv_file)
    sample_list = list(islice(p6, 10))
    custom_iter = TransactionLimitIterator(sample_list, max_count=2)
    print(f"\n[Custom Iterator] Перші записи через TransactionLimitIterator:")
    for item in custom_iter:
        print(f" * {item.category}: {item.amount} грн")

    # 7. Підсумкова статистика
    p7 = build_finance_pipeline(csv_file)
    stats = stream_financial_summary(p7)
    print("\n--- Фінансова статистика потоку ---")
    print(f"Доходи : {stats['total_income']:.2f} грн")
    print(f"Витрати : {stats['total_expense']:.2f} грн")
    print(f"Баланс : {stats['balance']:.2f} грн")

if __name__ == "__main__":
    main()
```

```

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  src/finance_stream/experiment.py
  src/finance_stream/generate_data.py
  src/finance_stream/main.py

nothing added to commit but untracked files present (use "git add" to track)

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git add .

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git commit -m "add dataset generator, benchmark experiment and entry point"
[main 53b483a] add dataset generator, benchmark experiment and entry point
3 files changed, 149 insertions(+)
create mode 100644 Lab03/src/finance_stream/experiment.py
create mode 100644 Lab03/src/finance_stream/generate_data.py
create mode 100644 Lab03/src/finance_stream/main.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git push origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 3.12 KiB | 3.12 MiB/s, done.
Total 8 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/AntoskovaA/paython.git
4ffe38c..53b483a main -> main

```

5. Встановлення пакета в режимі розробки (editable installation)

```

(.venv) PS C:\Users\Alina\Desktop\пайтон\python\Lab03> python -m pip install -e .
Obtaining file:///C:/Users/Alina/Desktop/%D0%BF%D0%B0%D0%B9%D1%82%D0%BE%D0%BD/paython/Lab03
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: finance-stream
Building editable for finance-stream (pyproject.toml) ... done
Created wheel for finance-stream: filename=finance_stream-0.1.0-0.editable-py3-none-any.whl
size=1612 sha256=f779e84ea2ce24f0528d1748398a822f24a11cb35d0813410da1c5870c44d571
Stored in directory: C:\Users\Alina\AppData\Local\Temp\pip-ephem-wheel-cache-cfa731pl\wheels\
Successfully built finance-stream
Installing collected packages: finance-stream
Successfully installed finance-stream-0.1.0

```

6. Демонстрація роботи програми: потоковий вивід операцій, робота генераторних виразів, батчинг та кумулятивний баланс

```

(.venv) PS C:\Users\Alina\Desktop\пайтон\python\Lab03> python -m finance_stream.main
=== Лабораторна робота № 3: Потоковий конвеєр (Варіант 12) ===

--- Перші 5 операцій (itertools.islice) ---
2026-09-02 | EXPENSE | Продукти | 1457.17 грн | Операція №1
2026-09-03 | EXPENSE | Комунальні | 656.89 грн | Операція №2
2026-09-04 | EXPENSE | Одяг | 1196.90 грн | Операція №3
2026-09-05 | EXPENSE | Транспорт | 3822.24 грн | Операція №4
2026-09-06 | EXPENSE | Продукти | 1648.99 грн | Операція №5

[Generator expression] Загальні витрати: 1407693.35 грн

[Filter & islice] Витрати на 'Продукти' >= 1500 грн (знайдено 3)

[Batching] Розмір першого батча: 250 записів

[itertools.accumulate] Кумулятивний баланс перших 5 операцій: [-1457.17, -2114.06, -3310.96, -7133.2, -8782.19]

[Custom Iterator] Перші записи через TransactionLimitIterator:
* Продукти: 1457.17 грн
* Комунальні: 656.89 грн

--- Фінансова статистика потоку ---
Доходи : 4496997.14 грн
Витрати : 1407693.35 грн
Баланс : 3089303.79 грн

```

7. Результати експериментального порівняння часу виконання та пікової пам'яті (tracemalloc) для Eager та Lazy варіантів

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab03> python -m finance_stream.experiment
```

Records	Eager time		Lazy time		Eager Peak MB		Lazy Peak MB
10000	0.1133	c	0.1527	c	4.82	MB	0.2179 MB
100000	1.2320	c	1.7102	c	48.21	MB	0.2191 MB
500000	5.3105	c	8.9633	c	242.12	MB	0.0664 MB

8. Верифікація стану Git-репозиторію

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git log --oneline
1f7623c (HEAD -> main, origin/main) updated to remove error
53b483a add dataset generator, benchmark experiment and entry point
4ffe38c implement stream processing generators, pipeline and itertools analytics
7d49f2c created virtual environment and models with custom iterator
3f648db initial commit, setup project configuration for Lab03
6de8c86 add report
df46e40 Lab02: add benchmark measurements and entrypoint
5ca9d97 Lab02: implement data structures, collections, closures and decorators
dffffb15 create src-layout package structure and modules
6cc5c15 Lab02: initial commit, setup project configuration
76258dc saved report as pdf
fc4c505 modified report
2310b6a added report
41d948a Add main entry point and README
74be2b5 Add transaction model and calculation services
ecb0c51 created folders for project, init files and virtual environment
2215b9f initial commit, setup project

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab03 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Висновок:

Під час виконання лабораторної роботи № 3 було опановано ітераційну модель Python, принципи лінивих обчислень (lazy evaluation) та механізми потокової обробки даних без повного завантаження датасетів у RAM.

На базі предметної області фінансового трекера (Варіант 12) реалізовано повноцінний потоковий конвеєр, що включає генераторне читання файлу (yield from), парсинг, валідацію, нормалізацію, розбиття на пакети (batching) та потокову агрегацію за допомогою інструментів модуля itertools (accumulate, islice). Також створено власний клас-ітератор TransactionLimitIterator відповідно до протоколу `__iter__` / `__next__`.

Проведений порівняльний бенчмарк (Eager vs Lazy) на обсягах даних до 500 000 записів за допомогою модуля tracemalloc практично підтвердив головну перевагу генераторів: підхід Eager демонструє лінійне зростання пікової пам'яті ($O(n)$) до понад 300 МБ, тоді як лінивий конвеєр утримує споживання оперативної пам'яті на стабільно низькому рівні ($O(1)$, менше 0.1 МБ) незалежно від розміру файлу. Розроблений проєкт організовано за принципом src-layout, встановлено в режимі редагування та зафіксовано в системі контролю версій Git.